

Step 1: Define the problem

I notice that the Asian Institute of Computer Studies has computing in its name, but its daily operations still run on paper. Students like me wait days for printed grade reports. Teachers manage scores in disconnected spreadsheets and retype them into forms at the end of every term. Administrators run a campus with no single view of its own students, teachers, rooms, and schedules (What I think). The exact problem is that gap between what the school teaches and how it operates. This project closes it with one web portal. Students get real-time grades, schedules, and documents instead of a printed queue. Teachers get grade entry that locks on submit and reaches the student dashboard directly. Administrators get branch-scoped oversight where each campus sees only itself. The users are the three roles the school already has, and the help is concrete: a student checks standing before the add-drop deadline, a teacher submits grades once, and an admin approves a request without walking to another office.

Step 2: Design the solution

Algorithm declaration

- Password authentication uses *bcrypt* with a cost factor of 12 and constant-time comparison to block timing attacks. Sessions are JSON web tokens in `httpOnly` cookies with a 7-day expiry, rotated on refresh.
- Face recognition runs *face-api.js* (That I just recently discover and think it's fun to try it out and play with it) locally with *TinyFaceDetector*, *FaceLandmark68Net*, and *FaceRecognitionNet*. It extracts 128-dimensional embeddings and compares the live embedding against stored ones using Euclidean distance with a 0.6 threshold. Enrollment stores the embedding only, never the raw image, and a failed match falls back to password login. The design has no liveness detection, so a photo can spoof it. I document that honestly as convenience auth, not security.
- Grade computation applies a weighted average where prelim counts 30 percent, midterm 30 percent, and finals 40 percent. Missing periods redistribute their weight. The final rounds to two decimals and maps to remarks from Excellent at 90 and above down to Failed below 70. (Still not sure if this the right computation, looking for further refinement)
- Schedule management uses a 6 by 9 matrix for Monday through Saturday, 8 AM to 4 PM. Conflict detection runs a sweep-line algorithm that sorts assignments by start time and tracks active teachers and rooms, flagging overlaps in one pass.
- Enrollment prerequisite checks run a topological sort over the curriculum graph and block a student missing any prerequisite with a final grade of 75 or higher.

- Capacity checks use an atomic counter on the subject document, decremented on enroll and incremented on drop, which stops two students from taking the last seat at once.
- Search uses MongoDB text indexes on names, emails, student IDs, and subject codes, with compound indexes on branch, role, and status for scoped lists.
- Audit logging is append-only. Every sensitive action writes one immutable document holding the actor, the target, before and after values, a timestamp, the IP address, and the user agent.
- Certificate of enrollment generation runs client-side with jsPDF and autotable. It fetches current-term enrollments, builds a table of codes, names, and units, totals the units, and adds a registrar signature line and issue date.
- Every operation reports results through one error convention. Success returns `ok true` with the data. Failure returns `ok false` with a code drawn from the fixed set `VALIDATION`, `NOT_FOUND`, `UNAUTHORIZED`, `DUPLICATE`, and `INTERNAL`.

```
try: return {ok:true, data: op()}
```

```
catch ValidationError: return {ok:false, code:"VALIDATION", msg}
```

```
catch NotFound: return {ok:false, code:"NOT_FOUND"}
```

```
catch Unauthorized: return {ok:false, code:"UNAUTHORIZED"}
```

```
catch Duplicate: return {ok:false, code:"DUPLICATE"}
```

```
catch *: log(err); return {ok:false, code:"INTERNAL"}
```

Data structures

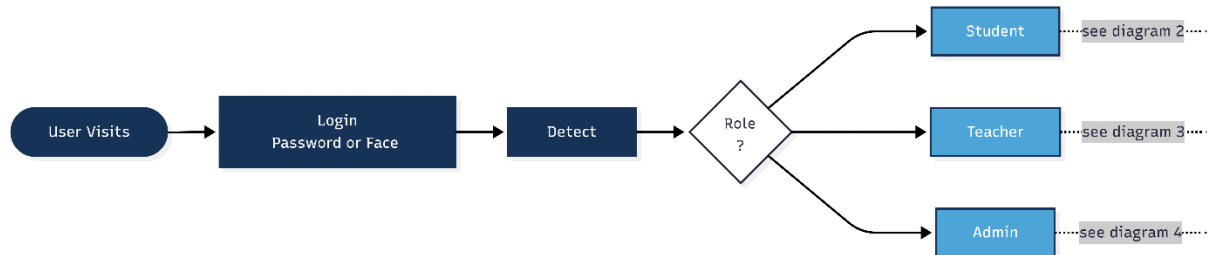
The database is MongoDB with branch scoping enforced at the query level. Every query injects the user branch ID automatically, so no cross-branch leak is possible there (Though I am considering to use cloudinary database for storing and using/uploading assets there to use instead of blatantly putting images within the code).

Core collections include:

- users: roles, branch IDs, hashed passwords, face embeddings, and status flags.
- branches: campus settings, academic years, semesters, and grading periods.
- courses and subjects: programs, units, prerequisites, and capacity limits.
- sections and schedules: teacher, room, and time-slot mappings plus temporary changes and conflict flags.
- enrollments and grades: student-to-subject links with period scores, lock states, and submission timestamps.
- rooms: capacity, equipment, and availability.
- documents: Cloudinary URLs for uploads with verification statuses.
- audit_logs: the append-only record of every sensitive action.
- notifications, announcements, and events: communication and the campus calendar.

- attendance and faculty_attendance: daily presence and in/out logs.
- requests, enrollment_periods, terms, and curriculum: approval workflows and the academic structure.

Step-by-step logic plans



Authentication flow:

1. User submits email and password
2. System finds the user and verifies the bcrypt hash.
3. The system rejects any user whose status is not active.
4. On success the system signs a token carrying the user ID, role, and branch ID.
5. The token sets an httpOnly cookie and redirects to the branch-scoped dashboard.
6. Face login captures a webcam frame, generates an embedding, and compares it against branch users. A match under 0.6 triggers the same token flow, and a miss falls back to the password form.

login(email, password):

```

user = findUser(email)

if not user or not verify(password, user.hash): fail

if user.status != "active": fail

token = signJWT({sub:user.id, role:user.role, branch:user.branchId})

setCookie(token); return {user, redirect: "/portal/"+user.branchId+"/"+user.role}
  
```

faceLogin(faceImage, branchId):

```

live = faceAPI.embed(faceImage)

for user in users(branchId, hasEmbedding=true):
    if distance(live, user.embedding) < 0.6:
        return loginAs(user) // convenience only, password is primary

fail → fallback to password
  
```

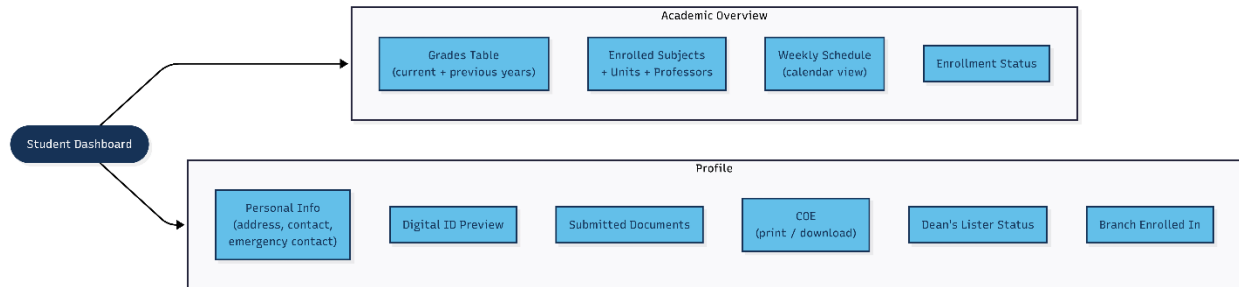
Branch scoping logic:

1. Every API handler reads the branch ID from the verified request
2. A query helper injects that ID into all find, update, and delete operations.

query(collection, filter):

```
return db[collection].find({...filter, branchId: currentUser.branchId})
```

Student dashboard logic:



1. The client fetches profile, current-term grades, the schedule grid, and today's tasks in parallel.
2. The grades query pulls enrollments, joins subject data, reads grade records, computes the weighted final, and assigns remarks.
3. The schedule query pulls active sections, paints them onto the 6 by 9 grid, and flags conflicts.
4. The tasks query checks enrolled subjects for pending or missing activities.

getGrades(studentId, term):

```
for each enrollment in enrollments(studentId, term):
    subject = getSubject(enrollment.subjectId)
    g = getGradeRecords(enrollment.id)
    final = round(0.3*g.prelim + 0.3*g.midterm + 0.4*g.final)
    yield {subject, units, teacher, prelim:g.p, mid:g.m, fin:g.f, final, remark(final)}
```

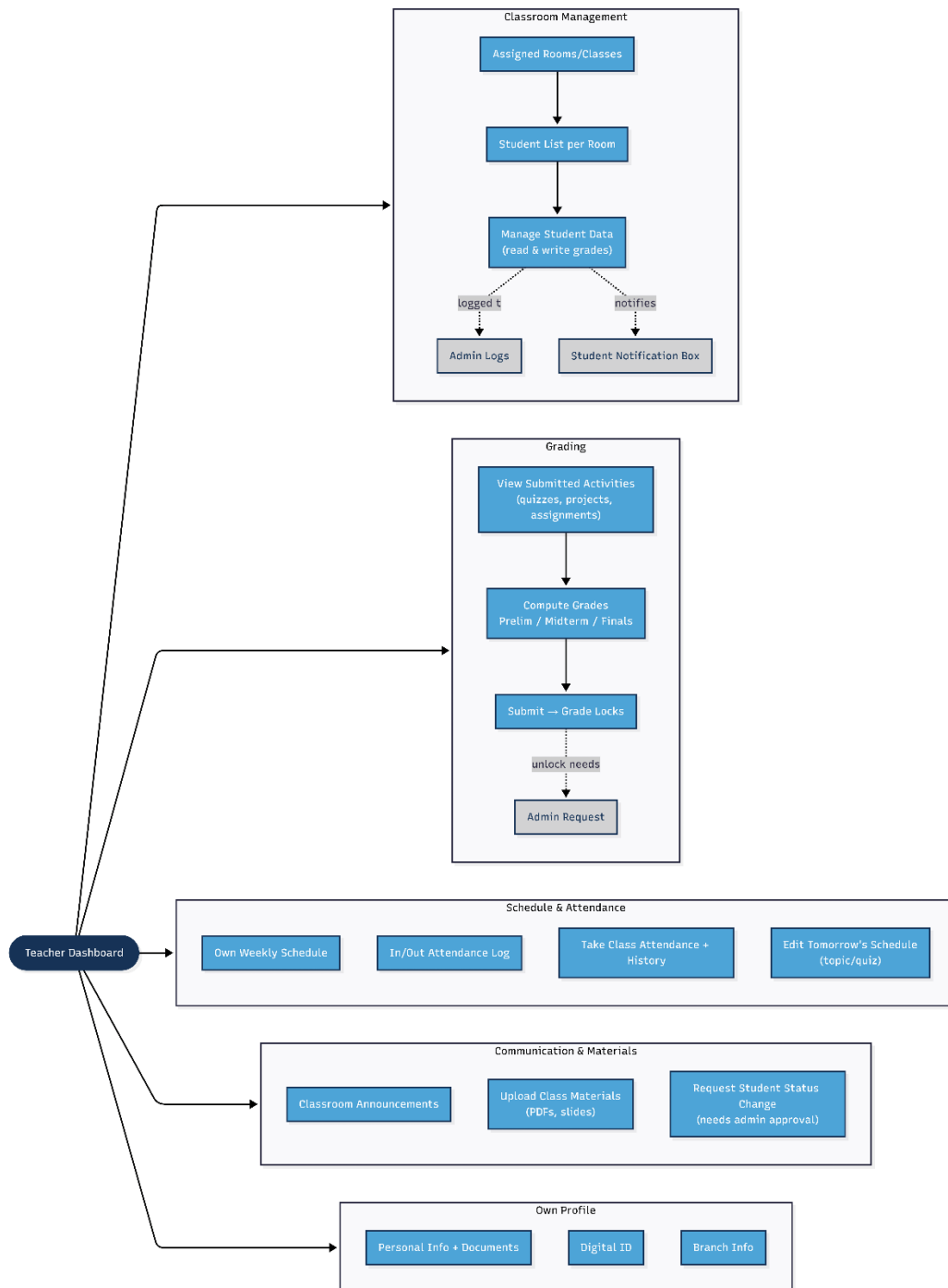
getSchedule(studentId, term):

```
grid[6 days][8..16hrs] = empty
for each active enrollment:
    for each class in section.schedule:
        grid[day][hour] = {subject, room, teacher} // mark conflicts
return grid
```

todaysClasses(studentId):

```
today = grid[dayOfWeek()][currentHour..]
return {current, upcoming, past}
```

Teacher grading logic:



1. Teacher opens a section and fetches the roster.
2. Teacher enters grades for prelim, midterm, or finals.
3. On submit the system verifies the teacher owns the section.
4. It bulk upserts the grades and sets the lock flag.
5. It writes an audit log and creates a notification for each affected student.
6. A locked grade changes only through an admin unlock request.

```
submitGrades(teacherId, sectionId, subjectId, period, entries):
```

```
    verify teacher owns section/subject
```

```
    if any grade locked: fail "contact admin"
```

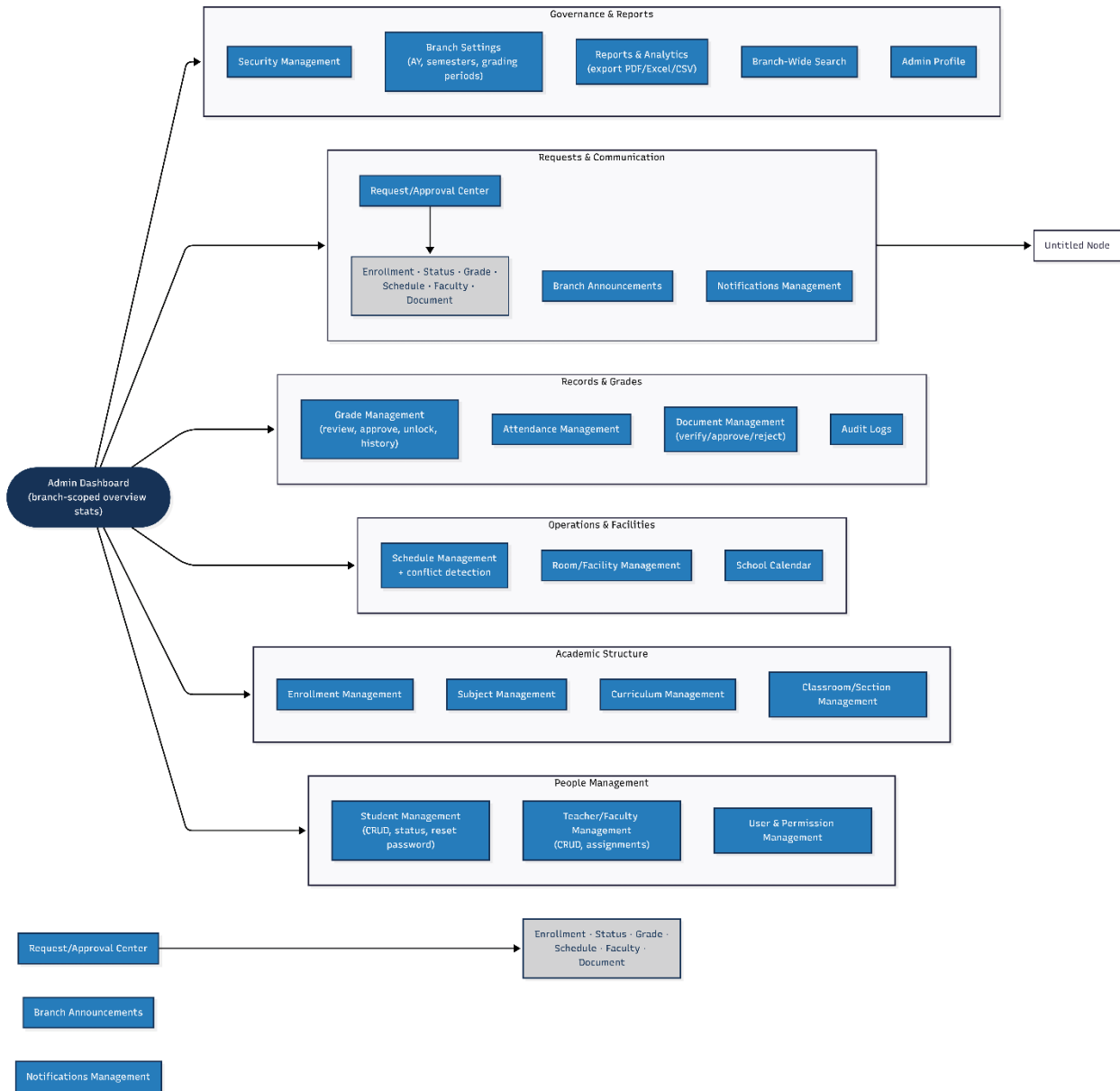
```
    bulk upsert grades {enrollmentId, period, value, by:teacherId}
```

```
    lock all grades for (section, subject, period)
```

```
    auditLog("GRADE_SUBMIT", teacherId, entries)
```

```
    notifyStudents(entries, "grade posted")
```

Admin dashboard logic:



1. The dashboard aggregates branch stats for students, teachers, active classes, and rooms.
2. Admin gets create, read, update, and delete interfaces for users, subjects, and sections
3. Admin manages enrollment periods and approves or rejects pending requests
4. Every mutation writes to the immutable audit log.

```
branchOverview(branchId):  
  
return {  
  
  students: count(users, role=student, active),  
  teachers: count(users, role=teacher, active),  
  classes: count(sections, active, currentTerm),  
  rooms: count(rooms, available),  
  pendingRequests, pendingDocs,  
  recent: auditLogs(limit=10)  
}
```

```
createStudent(branchId, data):  
  
user = createUser({...data, role:"student", branchId, hash:hash(tempPass)})  
autoEnroll(user, curriculum(course, year))  
auditLog("STUDENT_CREATE", adminId, user.id)  
return {userId, tempPassword}
```

Enrollment logic:

1. Admin opens a period by setting start and end dates.
2. A student request passes four checks in order: period open, prerequisites met, no schedule conflict, seat available.
3. A failed check returns its specific code so the student sees the reason.
4. A clean pass creates the enrollment and decrements the seat counter.

```
enrollSubject(studentId, subjectId):  
  
  if not enrollmentOpen(branch): fail  
  
  if prereqsNotMet(student, subject): fail  
  
  if hasConflict(student.schedule + subject): fail  
  
  if subject.full: fail  
  
  createEnrollment(studentId, subjectId, currentTerm)
```


Information flow outline

All traffic passes through central middleware. Unauthenticated requests land on the login page, and authenticated requests carry the user context and branch ID into every later call. The middleware protects all portal routes and redirects strangers to login. When a student loads the dashboard, the client fires parallel requests for grades, schedule, and profile. The server resolves them against the branch-scoped database and returns formatted JSON.

- Teachers work through the grading module. A submission locks the records and triggers the notification writes.
- Administrators configure enrollment windows, approve schedule changes, and verify uploaded documents. Uploads route through Cloudinary and the database stores only the secure URL. The certificate of enrollment runs the opposite direction. The client fetches enrollments and builds the PDF locally.

`generateCOE(studentId):`

```
student, enrollments = fetch...  
  
pdf = new PDF()  
  
pdf.text("CERTIFICATE OF ENROLLMENT")  
  
pdf.text(student.name, student.id, course, branch, term)  
  
pdf.table(enrollments → code, name, units)  
  
pdf.text("Total: " + sum(units))  
  
pdf.signatureLine(); pdf.date()  
  
return pdf.blob()
```

- Notifications (not 100% sure doing this, I just think of it and write it down. Need to find more information and thoughts about this.) write to their collection on any state change. A WebSocket server pushes to connected clients, and everyone else polls the notifications endpoint on page focus.

Every sensitive route sits inside the audit wrapper. On mutation it captures the before state, performs the write, captures the after state, and appends one log document. The collection accepts no updates and no deletes.

`log(action, actor, target, meta, oldVal, newVal):`

```
create audit_logs {action, actorId, actorRole, branchId, targetType, targetId,  
  previousValue:oldVal, newValue:newVal, meta, ip, ua, time:now()}
```

Reports run through MongoDB aggregation pipelines for population, grade distribution, and attendance rates. A serverless function formats the results and exports PDF, Excel, or CSV.

- Global search queries the text indexes across users, subjects, rooms, sections, and documents, groups results by type, and scopes them to the branch